케이녹 C언어 2회차 과제

17기 최동현 (202511688)

1-1. 제어문 (선택 제어문)

- **선택 제어문 (if 제어문)**

  : 선택 제어문 혹은 조건문이라고 불리는 제어문은 **여러 상황에 따라 다른 코드를 실행**하고 싶을 때 사용하는 문법입니다.

```
// 선택 제어문 (if 조건문)

if (조건식) {        // 해당 조건식이 참일때,
  코드();           // {} 안에 있는 코드를 실행.
} else if (조건식) {    // 위의 조건이 맞지 않고, else if 조건식이 참일 경우,
  코드();           // else if {} 안에 있는 코드를 실행.
} else {          // 위의 조건식이 전부 참이 아닐때,
  코드();           // else {} 안에 있는 코드를 실행.
}
```

1-1. 제어문 (선택 제어문)

- C언어 if 제어문 (예시 코드 화면)

```c
#include <stdio.h>

int main(void) {
    int isAdmin = 0;

    printf("서버에 접근합니다.\n");
    printf("접근 권한 선택: ");
    scanf("%d", &isAdmin);

    if (isAdmin == 1) {
        printf("관리자 권한으로 로그인 하셨습니다. (모든 데이터 열람 가능)\n");
    } else if (isAdmin == 2) {
        printf("부서 담당자로 로그인 하셨습니다. (일부 데이터 열람 가능)\n");
    } else {
        printf("일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.\n");
    }

    return 0;
}
```

1-1. 제어문 (선택 제어문)

- C언어 if 제어문 (코드 실행 화면)

```
100%                    13%                    15 GB
~/Desktop/CHOI   vim ./KKnock/Day2/main.c

서버에 접근합니다.
접근 권한 선택:1
관리자 권한으로 로그인 하셨습니다. (모든 데이터 열람 가능)

계속하려면 엔터 혹은 명령을 입력하십시오
서버에 접근합니다.
접근 권한 선택:2
부서 담당자로 로그인 하셨습니다. (일부 데이터 열람 가능)

계속하려면 엔터 혹은 명령을 입력하십시오
서버에 접근합니다.
접근 권한 선택:3
일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.

계속하려면 엔터 혹은 명령을 입력하십시오
서버에 접근합니다.
접근 권한 선택:0
일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.

계속하려면 엔터 혹은 명령을 입력하십시오
```

1-1. 제어문 (선택 제어문)

- C언어 if 제어문 (주의사항!)

```c
#include <stdio.h>

int main(void) {
    int isAdmin = 0;

    printf("서버에 접근합니다.\n");
    printf("접근 권한 선택: ");
    scanf("%d", &isAdmin);

    // isAdmin == 1 이 아닌 isAdmin = 1로 작성 (실수로)
    if (isAdmin = 1) {
        printf("관리자 권한으로 로그인 하셨습니다. (모든 데이터 열람 가능)\n");
    } else {
        printf("일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.\n");
    }

    return 0;
}
```

1-1. 제어문 (선택 제어문)

- C언어 if 제어문 (주의사항!)

```
100%                                  6%                                  15 GB
~/Desktop/CHOI   vim ./KKnock/Day2/main.c

./KKnock/Day2/main.c:11:17: warning: using the result of an assignment as a condition without par
   11 |     if (isAdmin = 1) {
      |         ~~~~~~~~~^~~
./KKnock/Day2/main.c:11:17: note: place parentheses around the assignment to silence this warning
   11 |     if (isAdmin = 1) {
      |                 ^
      |         (       )
./KKnock/Day2/main.c:11:17: note: use '==' to turn this assignment into an equality comparison
   11 |     if (isAdmin = 1) {
      |                 ^
      |                 ==
1 warning generated.
서버에 접근합니다.
접근 권한 선택: 0
관리자 권한으로 로그인 하셨습니다. (모든 데이터 열람 가능)

계속하려면 엔터 혹은 명령을 입력하십시오
```

★ if 제어문의 괄호 안 조건식 결과가 **0**이 아니면 **무조건 참(True)**로 인식하기 때문에 주의 필요!

1-2. 제어문 (선택 제어문)

- **선택 제어문 (switch-case 제어문)**

  : switch-case문도 if문처럼 조건을 나누는 목적은 같습니다. (그럼 왜 이 SW 제어문을 쓰는가?)
  선택을 하는 변수 값(정수나 문자)이 **명확한 값**을 가질 때 유용합니다. (ex. 자판기)

```
// 선택 제어문 (switch-case 제어문)

int 선택변수 = 1;   // 선택을 결정하는 변수

switch (선택변수) {      // 해당 선택변수를 기준으로 선택함.
  case 변수값_1:        // 변수값이 case 값과 일치하다면,
    코드();            // 해당 case의 코드 실행.
    break;            // case 실행 후, switch 블록을 빠져나간다.
  case 변수값_2:
    코드();
    break;

  ...

  default:       // 모든 case에 해당하지 않을 경우, (if의 else와 같음)
    코드();       // 해당 default case 코드 실행.
    break;       // 똑같이 switch 블록을 빠져나감.
}
```

switch-case가 if문 보다 빠른 이유 :

switch-case문은 if문과 다르게 컴파일러가 코드를 번역할 때,
점프 테이블(Jump Table)를 만들어 이동하기에 if문 보다 훨씬 빠르게 작동합니다. (**코드가 시작되는 메모리 주소로 이동**)

쉽게 예시를 들면 **if문**은 501층을 찾을때까지 1층부터 차근차근 찾습니다.
switch-case문은 엘리베이터가 이미 만들어져있어서 5층 버튼을 누르고 올라갑니다.

1-2. 제어문 (선택 제어문)

- C언어 switch-case 제어문 (예시 코드 화면)

```c
#include <stdio.h>

int main(void) {
    int userLevel = 0;

    printf("서버에 접근합니다.\n");
    printf("접근 권한 선택: ");
    scanf("%d", &userLevel);

    switch (userLevel) {
        case 1:
            printf("관리자 권한으로 로그인 하셨습니다. (모든 데이터 열람 가능)\n");
            break;
        case 2:
            printf("부서 담당자로 로그인 하셨습니다. (일부 데이터 열람 가능)\n");
            break;
        default:
            printf("일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.\n");
            break;
    }

    return 0;
}
```

1-2. 제어문 (선택 제어문)

- C언어 switch-case 제어문 (코드 실행 화면)

```
100%                                  13%                                  15 GB
~/Desktop/CHOI   vim ./KKnock/Day2/main.c

서버에 접근합니다.
접근 권한 선택:1
관리자 권한으로 로그인 하셨습니다. (모든 데이터 열람 가능)

계속하려면 엔터 혹은 명령을 입력하십시오
서버에 접근합니다.
접근 권한 선택:2
부서 담당자로 로그인 하셨습니다. (일부 데이터 열람 가능)

계속하려면 엔터 혹은 명령을 입력하십시오
서버에 접근합니다.
접근 권한 선택:3
일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.

계속하려면 엔터 혹은 명령을 입력하십시오
서버에 접근합니다.
접근 권한 선택:0
일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.

계속하려면 엔터 혹은 명령을 입력하십시오
```

1-2. 제어문 (선택 제어문)

- C언어 switch-case 제어문 (주의사항!)

```c
#include <stdio.h>

int main(void) {
    int userLevel = 0;

    printf("서버에 접근합니다.\n");
    printf("접근 권한 선택: ");
    scanf("%d", &userLevel);

    switch (userLevel) {
        case 1: // 1이면 일반유저
            printf("일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.\n");
            // break; 를 빼먹었을때 (개발자 실수)

        case 2: // 2이면 관리자
            printf("관리자 권한으로 로그인 하셨습니다. (모든 데이터 열람 가능)\n");
            break;
    }

    return 0;
}
```

1-2. 제어문 (선택 제어문)

- C언어 switch-case 제어문 (주의사항!)

```
100%                                                    6%
~/Desktop/CHOI   vim ./KKnock/Day2/main.c

서버에 접근합니다.
접근 권한 선택: 1   1은 일반유저
일반 유저로 로그인 하셨습니다. 접근이 거부되었습니다.
관리자 권한으로 로그인 하셨습니다. (모든 데이터 열람 가능)

계속하려면 엔터 혹은 명령을 입력하십시오
```

★ switch-case 제어문에서 실수로 **break를 빼먹었을 경우** 밑에 있는 case문들은 조건이 참이든 아니든 **break가 나올때까지** 코드들이 줄줄이 실행됩니다. 이를 전문용어로 **Fall-Through**(줄곧 떨어진다)라고 합니다. 그렇기에 주의 필요!

2-1. 제어문 (반복 제어문)

- **반복 제어문 (for 제어문)**

: 반복 제어문은 **똑같은 코드**를 **여러 번**(수백, 수천 등) **반복되어 실행**되어야 할 때 사용되는 문법입니다.

```
// 반복 제어문 (for 제어문 - 반복 횟수가 정해져 있을때 주로 사용)

for (초기식; 조건식; 증감식;) {
  코드();
}

// 이해를 돕기 위한 예시
for (int i = 0; i < 10; i++) { // 마지막 후위 증가연산자를 사용해서 i는 0으로 시작
  printf("%d ", i); // 0에서 9까지 연달아 반복 출력 (0 1 2 3 4 5 6 7 8 9 )
}

// 무한 반복
int 카운트 = 1;

for(;;) { // 초기식|조건식|증감식 아무것도 없이 ;;만 있다면 영원히 반복
  코드();
  // 밑의 조건식인 if문이 없다면 진짜 영원히 반복됩니다. (굳이 사용할 이유가..?)
  카운트++;
  if (카운트 == 5) { // 1부터 5까지 반복 후 break를 만나 for문 종료
    break;
  }
}
```

2-1. 제어문 (반복 제어문)

- C언어 for 제어문 (예시 코드 화면)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i;
    char command[20];   // 명령어(문자열)을 넣기위한 배열변수

    printf("폴더를 생성합니다.\n");

    for(i = 1; i <= 10; i++) {
        sprintf(command, "mkdir ./folder_%d", i);     // mkdir ./folder_1을 command 라는 배열 변수에 넣음
        // sprintf = String Print Formatted (형식에 맞춰 문자열 변수에 출력=넣기)
        system(command);        // mkdir forlder_1 ~ 10까지 실행.
        // system = 명령어 실행 함수 (stdlib.h 헤더 필요)
        printf("(%d) folder_%d 생성완료.\n", i, i);
    }

    printf("총 %d개의 폴더를 생성했습니다.\n", i - 1); // 마지막 1증가로 인해 -1해줌

    return 0;
}
```

2-1. 제어문 (반복 제어문)

- C언어 for 제어문 (코드 실행 화면)

```
100%                                  10%                                  14 GB
~/Desktop/CHOI/KKnock/Day2   vim ./main.c

폴더를 생성합니다.
(1) folder_1 생성완료.
(2) folder_2 생성완료.
(3) folder_3 생성완료.
(4) folder_4 생성완료.
(5) folder_5 생성완료.
(6) folder_6 생성완료.
(7) folder_7 생성완료.
(8) folder_8 생성완료.
(9) folder_9 생성완료.
(10) folder_10 생성완료.
총 10개의 폴더를 생성했습니다.

계속하려면 엔터 혹은 명령을 입력하십시오

~/Desktop/CHOI/KKnock/Day2   ls
Calculate    folder_1    folder_2    folder_4    folder_6    folder_8    main
Calculate.c  folder_10   folder_3    folder_5    folder_7    folder_9    main.c
```

순식간에 폴더 10개 생성 완료!

2-2. 제어문 (반복 제어문)

• 반복 제어문 (while 및 do-while 제어문)

: while문 같은 경우 **반복 횟수를 정확히 알지 못할 때**, 따로 **조건을 걸어** 해당 조건이 **참인 동안만 반복**할때 주로 씁니다. do-while문은 while과 똑같은 기능을 하지만 **조건에 상관없이 1번**은 무조건 **실행**됩니다.

```
// 반복 제어문 (while 제어문 - 반복 횟수가 정해져 있지 않을때 주로 사용)

while (조건식) {
  코드();
}

// 예시
int 조건 = 1;

while (조건 <= 5) { // 코드를 5번 실행
  코드();
  조건++; // 증가연산자가 없다면 조건은 계속 1이 되므로 무한 반복됩니다.
}
```

```
// 반복 제어문 (do-while 제어문 - while과 같음, 1번은 무조건 실행)

do {
  코드(); //  코드가 무조건 한번은 실행됩니다.
} while (조건식);
```

2-2. 제어문 (반복 제어문)

- C언어 while & do-while 제어문 (예시 코드 및 실행 화면)

```c
#include <stdio.h>

int main(void) {
    int x = 1;

    while (x <= 10) {
        printf("%d ", x);
        x++;
    }
    printf("\n");

    x = 10;

    do {
        printf("%d ", x);
    } while (x <= 5); // 조건이 맞지 않음 (10 <= 5)

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

1 2 3 4 5 6 7 8 9 10
10
계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 제어문 (흐름 제어문)

- **흐름 제어문 (break, continue, return, goto 제어문)**

  : 흐름 제어문이란 제어문의 흐름을 중간에 강제로 끊어버리거나, 다른 곳으로 점프 시키는 문법입니다.

```
// 흐름 제어문 (break, continue, return, goto)

break;      // 제어문의 흐름을 끊어버림
continue;   // 제어문에서 건너뛰기 역할
return;     // 함수 호출 및 결과 반환 (함수 입장에서 자신을 호출한곳으로 값을 가지고 반환)
goto;       // ~로 가다. 라는 말처럼 특정 위치(라벨)로 코드를 이동 시킵니다.
```

3. 제어문 (흐름 제어문)

- 흐름 제어문 (break 제어문 코드)

```c
#include <stdio.h>

int main(void) {
    for(int i = 1; i <= 10; i++) { // 1 ~ 10까지 10번 반복
        printf("%d\n", i); // i값 출력
        if (i == 5) break; // i값이 5라면 반복문 탈출 (1 ~ 5까지만 출력됨)
    }
    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

1
2
3
4
5

계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 제어문 (흐름 제어문)

- 흐름 제어문 (continue 제어문 코드)

```c
#include <stdio.h>

int main(void) {
    for(int i = 1; i <= 5; i++) { // 1 ~ 5까지 5번 반복
        if (i == 3) continue; // i값이 3이라면 밑의 코드 건너뛰기 (1 2 4 5만 출력됨)
        printf("%d\n", i); // i값 출력
    }
    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

1
2
4
5

계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 제어문 (흐름 제어문)

- 흐름 제어문 (return 제어문 코드)

```c
#include <stdio.h>

int main(void) {
    int sel = 0;

    printf("프로그램 실행되었습니다.\n리턴 값 입력: ");
    scanf("%d", &sel);

    if (sel == 1) { // sel이 1이라면
        return 1;   // main함수 리턴값 1 반환.
    } else if (sel == 10) {
        return 10; // 10반환
    } else {
        return 0;
    }
}
```

```
~/Desktop/CHOI  ./KKnock/Day2/main
프로그램 실행되었습니다.
리턴 값 입력: 0

~/Desktop/CHOI  echo "메인 함수 리턴값: $?"
메인 함수 리턴값: 0

~/Desktop/CHOI  ./KKnock/Day2/main
프로그램 실행되었습니다.
리턴 값 입력: 1

~/Desktop/CHOI  echo "메인 함수 리턴값: $?"
메인 함수 리턴값: 1

~/Desktop/CHOI  ./KKnock/Day2/main
프로그램 실행되었습니다.
리턴 값 입력: 10

~/Desktop/CHOI  echo "메인 함수 리턴값: $?"
메인 함수 리턴값: 10

~/Desktop/CHOI
```

3. 제어문 (흐름 제어문)

- 흐름 제어문 (main 함수 반환값 의미)

main함수 return값	의미
0	정상 종료
-1	에러 발생
1 이상	정상 종료됐으나, 다른 인자가 있음을 나타냄
-2 이하	에러 발생했고, 구체적으로 어떤 에러인지 나타냄

3. 제어문 (흐름 제어문)

- 흐름 제어문 (goto 제어문 코드)

```c
#include <stdio.h>

int main(void) {
    printf("1번 출력\n");
    printf("2번 출력\n");
    goto kknock;
    printf("3번 출력\n");

kknock:
    printf("K-Knock!\n");

    return 0;
}
```

goto의 단점 :

1. goto문은 if나 for처럼 {}블록 안에 있지 않습니다. (가독성 저하)
2. 절차 지향인 C에서 위로 아래로 막 이동하면 변수가 선언 되기도 전에 사용(쓰레기값 들어감)하게 되버리는 경우도 생깁니다. (변수 초기화 스킵)
3. 메모리를 할당 받아 쓰는 코드에서 goto를 쓰면 메모리 반납을 건너뛸 가능성이 있습니다. (리소스 누수)
4. 디버깅 지옥... (goto로 이동하는 코드가 아주 길면 에러를 찾는데 고생)

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

1번 출력
2번 출력
K-Knock!

계속하려면 엔터 혹은 명령을 입력하십시오
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 A+B 문제

문제

두 정수 A와 B를 입력받은 다음, A+B를 출력하는 프로그램을 작성하시오.

입력

첫째 줄에 A와 B가 주어진다. (0 < A, B < 10)

출력

첫째 줄에 A+B를 출력한다.

예제 입력 1 복사	예제 출력 1 복사
1 2	3

4. 실습 과제 (백준)

- 새싹 입력과 계산 A+B 풀이

```c
// 백준 새싹 입력과 계산
// A+B 문제

#include <stdio.h>

int main() {
    int a = 0, b = 0;

    scanf("%d %d", &a, &b);
    printf("%d\n", a + b);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

1 2
3

계속하려면 엔터 혹은 명령을 입력하십시오
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 A-B 문제

문제

두 정수 A와 B를 입력받은 다음, A-B를 출력하는 프로그램을 작성하시오.

입력

첫째 줄에 A와 B가 주어진다. (0 < A, B < 10)

출력

첫째 줄에 A-B를 출력한다.

예제 입력 1 복사

```
3 2
```

예제 출력 1 복사

```
1
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 A-B 풀이

```c
// 백준 새싹 입력과 계산
// A-B 문제

#include <stdio.h>

int main() {
    int a = 0, b = 0;

    scanf("%d %d", &a, &b);
    printf("%d\n", a - b);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

3 2
1

계속하려면 엔터 혹은 명령을 입력하십시오
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 A X B 문제

문제

두 정수 A와 B를 입력받은 다음, A×B를 출력하는 프로그램을 작성하시오.

입력

첫째 줄에 A와 B가 주어진다. (0 < A, B < 10)

출력

첫째 줄에 A×B를 출력한다.

예제 입력 1 복사

```
1 2
```

예제 출력 1 복사

```
2
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 A X B 풀이

```c
// 백준 새싹 입력과 계산
// A X B 문제

#include <stdio.h>

int main() {
    int a = 0, b = 0;

    scanf("%d %d", &a, &b);
    printf("%d\n", a * b);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

1 2
2

계속하려면 엔터 혹은 명령을 입력하십시오
3 4
12

계속하려면 엔터 혹은 명령을 입력하십시오
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 사칙연산 문제

문제

두 자연수 A와 B가 주어진다. 이때, A+B, A-B, A*B, A/B(몫), A%B(나머지)를 출력하는 프로그램을 작성하시오.

입력

두 자연수 A와 B가 주어진다. (1 ≤ A, B ≤ 10,000)

출력

첫째 줄에 A+B, 둘째 줄에 A-B, 셋째 줄에 A*B, 넷째 줄에 A/B, 다섯째 줄에 A%B를 출력한다.

예제 입력 1 복사

```
7 3
```

예제 출력 1 복사

```
10
4
21
2
1
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 사칙연산 풀이

```c
// 백준 새싹 입력과 계산
// 사칙연산 문제

#include <stdio.h>

int main() {
    int a = 0, b = 0;

    scanf("%d %d", &a, &b);
    printf("%d\n", a + b);
    printf("%d\n", a - b);
    printf("%d\n", a * b);
    printf("%d\n", a / b);
    printf("%d\n", a % b);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

7 3
10
4
21
2
1

계속하려면 엔터 혹은 명령을 입력하십시오
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 A / B 문제

문제

두 정수 A와 B를 입력받은 다음, A/B를 출력하는 프로그램을 작성하시오.

입력

첫째 줄에 A와 B가 주어진다. (0 < A, B < 10)

출력

첫째 줄에 A/B를 출력한다. 실제 정답과 출력값의 절대오차 또는 상대오차가 10^{-9} 이하이면 정답이다.

예제 입력 1 복사

```
1 3
```

예제 출력 1 복사

```
0.333333333333333333333333333333333
```

10^{-9} 이하의 오차를 허용한다는 말은 꼭 소수 9번째 자리까지만 출력하라는 뜻이 아니다.

4. 실습 과제 (백준)

- 새싹 입력과 계산 A / B 풀이

```c
// 백준 새싹 입력과 계산
// A / B 문제

#include <stdio.h>

int main() {
    int a = 0, b = 0;

    scanf("%d %d", &a, &b);
    printf("%.9lf\n", (double)a / b);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

1 3
0.333333333

계속하려면 엔터 혹은 명령을 입력하십시오
```

4. 실습 과제 (백준)

- 새싹 입력과 계산 꼬마정민 문제

문제

꼬마 정민이는 이제 A + B 정도는 쉽게 계산할 수 있다. 이제 A + B + C를 계산할 차례이다!

입력

첫 번째 줄에 A, B, C ($1 \le A, B, C \le 10^{12}$)이 공백을 사이에 두고 주어진다.

출력

A+B+C의 값을 출력한다.

예제 입력 1 복사	예제 출력 1 복사
77 77 7777	7931

4. 실습 과제 (백준)

- 새싹 입력과 계산 꼬마정민 풀이

```c
// 백준 새싹 입력과 계산
// 꼬마 정민 문제

#include <stdio.h>

int main() {
    long long int a = 0, b = 0, c= 0;

    scanf("%lld %lld %lld", &a, &b, &c);
    printf("%lld\n", a + b + c);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day2/main.c

77 77 7777
7931

계속하려면 엔터 혹은 명령을 입력하십시오
```

감사합니다

17기 최동현